

Warden Documentation

Process Manager · README & PM2 Comparison

Generated 2026-05-31

Contents

1. Warden — Internal Process Manager (README)
2. PM2 vs Warden — Side-by-Side Comparison

Warden — Internal Process Manager

A self-hosted, PM2-equivalent process manager built in TypeScript for Node 18+.

Manages **any** process (Node.js, Python, Ruby, binaries, shell scripts) with the full PM2 feature set and first-class telemetry.

Features

Category	Feature
Process modes	Fork mode (any process), Node.js Cluster mode (multi-instance, port-sharing)
Auto-restart	Crash detection, exponential backoff, <code>max_restarts</code> , <code>min_uptime</code> stability gate
Zero-downtime reload	Rolling reload for cluster apps (<code>warden reload</code>)
Scale	<code>warden scale <name> <n></code> — add/remove cluster workers at runtime
Watch	File-system watch triggers reload (<code>--watch</code>)
Cron restart	Scheduled restarts via cron expression (<code>--cron "0 0 * * *"</code>)
Memory limit	Auto-restart on memory threshold (<code>--max-memory-restart 150M</code>)
Log management	Per-process stdout/stderr files, rotation, streaming via <code>warden logs</code>
Persistence	<code>warden save</code> / <code>warden resurrect</code> with <code>~/.warden/dump.json</code>
Startup	Generate systemd / launchd service via <code>warden startup</code>
Ecosystem config	Declare multiple apps in <code>ecosystem.config.js</code> / <code>.json</code>
Live dashboard	<code>warden monit</code> — auto-refreshing terminal table
HTTP API	<code>GET /processes</code> , <code>/summary</code> , <code>/processes/:id/metrics</code>
Prometheus	<code>GET /metrics</code> endpoint (prom-client gauges/counters)
Telemetry	SQLite time-series, 24-hour retention, 5-second sampling

Installation

```
# From npm (global CLI):
npm install -g @roshan-gamage/warden

# From source:
git clone https://github.com/RoshanGamage01/warden.git
cd warden
npm install
npm run build
npm link # optional: link `warden` globally
```

The `warden` binary is at `./dist/bin/warden.js` after build.

Linux (Ubuntu/Debian): Prebuilt binaries are included for Node 20–24. If install still compiles from source and fails with `not found: make`, install build tools first:

```
sudo apt-get update && sudo apt-get install -y build-essential python3
```

Quick start

```
# Start any Node script directly
warden start ./server.js --name api

# Start a Python script
warden start ./worker.py --interpreter python3 --name worker

# Start with 4 cluster instances (Node.js only, port-sharing)
warden start ./app.js --instances 4 --name app

# Use an ecosystem config (ecosystem.config.js, warden.config.json, or *.config.js)
warden start ecosystem.config.js

# List all processes
warden list

# Stream logs
warden logs           # all processes
warden logs api       # by name
warden logs 0         # by id

# Live dashboard
warden monit
```

CLI Reference

Process management

Command	Description
<code>warden start <script></code>	Start a process (<code>.js</code> , <code>.py</code> , binaries, etc.)
<code>warden start <ecosystem.config.js></code>	Start all apps in an ecosystem config
<code>warden stop <id name></code>	Stop a process
<code>warden restart <id name></code>	Restart a process
<code>warden reload <id name></code>	Zero-downtime reload (cluster) or graceful restart (fork)
<code>warden delete <id name></code>	Stop and remove from registry
<code>warden scale <id name> <n></code>	Scale cluster app to N workers
<code>warden reset <id name></code>	Reset restart counters

Inspection

Command	Description
<code>warden list / warden ls</code>	Process table with CPU/mem/uptime
<code>warden describe <id name></code>	Full process detail including cluster workers
<code>warden monit</code>	Live refreshing dashboard (Ctrl+C to quit)
<code>warden ping</code>	Check daemon status

Logs

Command	Description
<code>warden logs [id]</code>	Stream logs (Ctrl+C to stop)
<code>warden logs -n 50 [id]</code>	Show last 50 historical lines then stream
<code>warden flush [id]</code>	Truncate log files

Persistence & startup

Command	Description
<code>warden save</code>	Write current process list to <code>~/.warden/dump.json</code>
<code>warden resurrect</code>	Start all saved processes
<code>warden startup [platform]</code>	Install system startup service (auto, systemd, launchd)
<code>warden unstartup [platform]</code>	Remove startup service

Daemon

Command	Description
<code>warden kill</code>	Gracefully stop all processes and the daemon

start options

```
-n, --name <name>           App name (defaults to script filename)
-i, --instances <n|max>     Number of instances; "max" uses all CPUs
    --exec-mode <fork|cluster> Execution mode (auto-set to cluster if -i > 1)
    --interpreter <binary>   Interpreter (e.g. python3, ruby)
    --node-args <args>       Extra Node.js args
    --cwd <dir>              Working directory
-e, --env <KEY=VAL>         Environment variable (repeatable)
    --watch                  Restart on file changes
    --max-memory-restart <size> Restart threshold (e.g. 150M, 1G)
    --cron <expr>           Scheduled restart (cron expression)
    --max-restarts <n>       Max crash-restart attempts (default 16)
    --min-uptime <time>     Stability gate (e.g. 1s, 500ms)
    --restart-delay <ms>    Delay between restarts
    --no-auto-restart        Disable crash auto-restart
    --merge-logs             Merge stdout and stderr into one file
    --kill-timeout <ms>     Graceful shutdown timeout
```

Ecosystem config

```
// ecosystem.config.js
module.exports = {
  apps: [
    {
      name: 'api',
      script: './src/api.js',
      instances: 2,
      exec_mode: 'cluster',
      env: { NODE_ENV: 'production', PORT: '3000' },
      max_memory_restart: '500M',
    },
    {
      name: 'worker',
      script: './src/worker.js',
      watch: ['./src'],
      ignore_watch: ['node_modules'],
      cron_restart: '0 3 * * *', // restart daily at 3am
    },
    {
      name: 'ml-model',
      script: './predict.py',
      interpreter: 'python3',
      autorestart: true,
      max_restarts: 5,
    },
  ],
};
```

```
warden start ecosystem.config.js
```

HTTP API

Runs on `http://127.0.0.1:9615` by default.

Endpoint	Description
<code>GET /health</code>	<code>{ status: 'ok', pid }</code>
<code>GET /processes</code>	All processes with latest CPU/mem
<code>GET /processes/:id</code>	Single process + 120 recent metric points
<code>GET /processes/:id/metrics?range=1h</code>	Time-series data (range: 1h , 6h , 24h , 7d)
<code>GET /summary</code>	Aggregate totals (online, stopped, errored, CPU, memory)
<code>GET /metrics</code>	Prometheus exposition format

Prometheus / Grafana

Point your Prometheus scrape config at `http://127.0.0.1:9615/metrics` :

```
# prometheus.yml
scrape_configs:
  - job_name: warden
    static_configs:
      - targets: ['127.0.0.1:9615']
```

Available metrics:

- `warden_process_cpu_percent{warden_id, name}`
 - `warden_process_memory_bytes{warden_id, name}`
 - `warden_process_restarts_total{warden_id, name}`
 - `warden_process_uptime_seconds{warden_id, name}`
 - `warden_process_online{warden_id, name, status}`
 - Standard Node.js process metrics via `prom-client`
-

File locations

Path	Purpose
<code>~/.warden/daemon.sock</code>	Unix domain socket (IPC)
<code>~/.warden/daemon.pid</code>	Daemon PID file
<code>~/.warden/dump.json</code>	Saved process list
<code>~/.warden/metrics.db</code>	SQLite metrics database (24-hour rolling window)
<code>~/.warden/logs/<name>--<id>/out.log</code>	Stdout log
<code>~/.warden/logs/<name>--<id>/err.log</code>	Stderr log
<code>~/.warden/logs/<name>--<id>/out.log.1</code>	Rotated log (up to 5 backups)

Architecture overview

```
CLI (warden)
├─ JSON-RPC over Unix domain socket
└─ ▼
  Daemon (God process)
  ├── IPC Server      ─ RPC request router
  ├── AppRegistry    ─ Registry of all managed apps
  │   ├── ProcessSupervisor ─ fork-mode lifecycle, backoff, log piping
  │   └── ClusterManager  ─ fork(cluster/master.js) → N workers
  ├── WatchManager   ─ chokidar → restart
  ├── CronScheduler  ─ croner → restart
  ├── MemoryMonitor  ─ pidusage → restart on threshold
  ├── MetricsSampler ─ pidusage every 5s → SQLite
  ├── LogRotator     ─ size-based rotation every 60s
  └── HTTP API (Fastify) ─ /processes, /metrics (Prometheus)
```

Daemon bootstrap

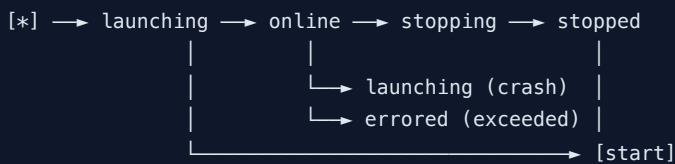
1. CLI checks if `~/.warden/daemon.sock` is connectable
2. If not, spawns daemon detached: `node dist/daemon/index.js`
3. CLI polls socket every 100 ms (max 5 s) for readiness
4. CLI connects and sends JSON-RPC

IPC framing

```
[4-byte BE uint32: body length] [UTF-8 JSON payload]
```

Handles TCP fragmentation and multiple messages per chunk.

Process state machine



Development

```
npm run dev      # watch mode build
npm test         # unit tests
npm run test:watch # watch mode

# Run integration tests (starts/stops real daemon)
npm test         # integration tests are auto-run (uses isolated ~/.warden)

# Skip integration tests
SKIP_INTEGRATION=1 npm test
```

Windows support

On Windows, the Unix domain socket path is replaced with a named pipe (`\\.\pipe\warden-daemon`). All other functionality is identical. The `pidusage` and `better-sqlite3` packages are cross-platform. If prebuilt binaries are not available for your Node version, native compilation may be required — on Windows use [Build Tools for Visual Studio](#); on Linux use `build-essential` (see Installation above).

Environment variables

Variable	Default	Description
<code>WARDEN_API_PORT</code>	<code>9615</code>	HTTP API port
<code>WARDEN_API_HOST</code>	<code>127.0.0.1</code>	HTTP API bind address

License

MIT

PM2 vs Warden — Side-by-Side Comparison

Warden is a self-hosted process manager built as a PM2-equivalent for Node 18+. This document compares the two tools feature-by-feature so you can decide which fits your stack — or migrate from PM2 with minimal friction.

At a glance

	PM2	Warden
Purpose	Production process manager for Node.js and beyond	Self-hosted PM2 alternative with built-in telemetry
Language	JavaScript	TypeScript
License	AGPL-3.0	MIT
Node requirement	Node 12+	Node 18+
Install	<code>npm install -g pm2</code>	<code>npm install -g @roshan-gamage/warden</code>
CLI binary	<code>pm2</code>	<code>warden</code>
Daemon IPC	Unix socket / TCP	Unix domain socket (named pipe on Windows)
Config home	<code>~/.pm2/</code>	<code>~/.warden/</code>
HTTP API	Optional (via PM2 Plus / custom)	Built-in on <code>127.0.0.1:9615</code>
Prometheus	Via third-party modules	Native <code>/metrics</code> endpoint
Metrics storage	External (PM2 Plus cloud)	Local SQLite (24-hour rolling window)
Cloud dashboard	PM2 Plus (paid)	None — self-hosted only

Feature comparison

Feature	PM2	Warden	Notes
Fork mode (any process)	✓	✓	Node, Python, Ruby, binaries, shell scripts
Cluster mode (Node.js)	✓	✓	Multi-instance, shared port
Auto-restart on crash	✓	✓	Exponential backoff in Warden
Max restarts / min uptime	✓	✓	Stability gate before resetting crash counter
Zero-downtime reload	✓	✓	Cluster: rolling reload; fork: graceful restart
Runtime scale	✓	✓	<code>pm2 scale</code> ↔ <code>warden scale</code>
File watch → reload	✓	✓	chokidar-based in Warden
Cron restart	✓	✓	Scheduled restarts via cron expression
Memory limit restart	✓	✓	e.g. <code>--max-memory-restart 150M</code>
Per-process logs	✓	✓	stdout / stderr to separate files
Log streaming	✓	✓	<code>pm2 logs</code> ↔ <code>warden logs</code>
Log rotation	✓ (module)	✓ (built-in)	Warden rotates on size; PM2 uses <code>pm2-logrotate</code> module
Log flush	✓	✓	<code>pm2 flush</code> ↔ <code>warden flush</code>
Save / resurrect	✓	✓	<code>dump.pm2</code> ↔ <code>dump.json</code>
Startup hook	✓	✓	systemd, launchd; PM2 also supports Windows service via extra package
Ecosystem config	✓	✓	<code>ecosystem.config.js</code> — largely compatible field names
Live terminal dashboard	✓ <code>pm2 monit</code>	✓ <code>warden monit</code>	Auto-refreshing CPU / memory table
Process list / describe	✓	✓	<code>pm2 list</code> / <code>pm2 show</code> ↔ <code>warden list</code> / <code>warden describe</code>
Reset restart counters	✓	✓	<code>pm2 reset</code> ↔ <code>warden reset</code>
Graceful daemon shutdown	✓ <code>pm2 kill</code>	✓ <code>warden kill</code>	Stops all apps and the god process
Environment variables	✓	✓	CLI <code>-e KEY=VAL</code> or config <code>env</code> block
Custom interpreter	✓	✓	<code>--interpreter python3</code> etc.

Feature	PM2	Warden	Notes
Merge logs	✓	✓	Single combined stdout/stderr file
Kill timeout	✓	✓	Graceful shutdown before SIGKILL
HTTP REST API	⚠ Limited	✓ Built-in	Warden: <code>/processes</code> , <code>/summary</code> , <code>/health</code>
Prometheus metrics	⚠ Via modules	✓ Native	Warden exposes <code>warden_process_*</code> gauges
Time-series metrics DB	✗	✓	SQLite, 5 s sampling, 24 h retention
PM2 Plus cloud monitoring	✓ (paid)	✗	Alerts, web UI, multi-server — PM2 only
Module ecosystem	✓	✗	<code>pm2-logrotate</code> , <code>pm2-server-monit</code> , etc.
Deploy automation	✓ <code>pm2 deploy</code>	✗	Git-based deployment workflow — PM2 only
Process attach / debug	✓ <code>pm2 attach</code>	✗	Attach to process stdio — PM2 only
Custom triggers / hooks	✓ <code>pm2 trigger</code>	✗	Event-driven custom actions — PM2 only
Docker official image	✓	✗	PM2 publishes maintained Docker images
Windows service	✓ (via <code>pm2-windows-service</code>)	⚠ Partial	Warden runs on Windows; no native service installer
JSON machine output	✓ <code>pm2 jlist</code>	✗	Scriptable JSON output — PM2 only
In-place PM2 upgrade	✓ <code>pm2 update</code>	✗	Reinstall via npm for Warden
Multi-server fleet mgmt	✓ (PM2 Plus)	✗	Centralized fleet view — PM2 Plus only

Legend: ✓ Supported · ⚠ Partial or requires extra setup · ✗ Not supported

CLI command mapping

Most day-to-day PM2 commands have a direct Warden equivalent:

PM2	Warden	Description
<code>pm2 start app.js</code>	<code>warden start app.js</code>	Start a process
<code>pm2 start ecosystem.config.js</code>	<code>warden start ecosystem.config.js</code>	Start from ecosystem file
<code>pm2 stop <id name></code>	<code>warden stop <id name></code>	Stop a process
<code>pm2 restart <id name></code>	<code>warden restart <id name></code>	Hard restart
<code>pm2 reload <id name></code>	<code>warden reload <id name></code>	Zero-downtime reload (cluster)
<code>pm2 delete <id name></code>	<code>warden delete <id name></code>	Stop and remove from registry
<code>pm2 scale <name> <n></code>	<code>warden scale <name> <n></code>	Scale cluster workers
<code>pm2 list / pm2 ls</code>	<code>warden list / warden ls</code>	Process table
<code>pm2 show <id name></code>	<code>warden describe <id name></code>	Detailed process info
<code>pm2 logs [id]</code>	<code>warden logs [id]</code>	Stream logs
<code>pm2 flush [id]</code>	<code>warden flush [id]</code>	Truncate log files
<code>pm2 monit</code>	<code>warden monit</code>	Live dashboard
<code>pm2 save</code>	<code>warden save</code>	Persist process list
<code>pm2 resurrect</code>	<code>warden resurrect</code>	Restore saved processes
<code>pm2 startup</code>	<code>warden startup</code>	Install boot-time service
<code>pm2 unstartup</code>	<code>warden unstartup</code>	Remove boot-time service
<code>pm2 reset <id name></code>	<code>warden reset <id name></code>	Reset restart counters
<code>pm2 kill</code>	<code>warden kill</code>	Shut down daemon and all apps
—	<code>warden ping</code>	Check daemon status (no PM2 equivalent)

PM2 commands with no Warden equivalent

PM2 command	What it does
<code>pm2 deploy</code>	Git-based deployment to remote servers
<code>pm2 attach</code>	Attach to a running process's stdio
<code>pm2 trigger</code>	Fire custom event hooks
<code>pm2 sendSignal</code>	Send arbitrary signals to a process
<code>pm2 jlist / pm2 prettylist</code>	JSON output for scripting
<code>pm2 env <id></code>	Show resolved environment for a process
<code>pm2 update</code>	In-place upgrade of PM2 itself
<code>pm2 install <module></code>	Install PM2 modules

start options comparison

Option	PM2	Warden
<code>--name</code>	✓	✓ <code>-n, --name</code>
<code>--instances / -i</code>	✓	✓ <code>-i, --instances (supports max)</code>
<code>--exec-mode</code>	✓	✓ <code>--exec-mode fork cluster</code>
<code>--interpreter</code>	✓	✓ <code>--interpreter</code>
<code>--node-args</code>	✓	✓ <code>--node-args</code>
<code>--cwd</code>	✓	✓ <code>--cwd</code>
<code>--env / -e</code>	✓	✓ <code>-e, --env KEY=VAL</code>
<code>--watch</code>	✓	✓ <code>--watch</code>
<code>--max-memory-restart</code>	✓	✓ <code>--max-memory-restart</code>
<code>--cron / cron_restart</code>	✓	✓ <code>--cron</code>
<code>--max-restarts</code>	✓	✓ <code>--max-restarts</code>
<code>--min-uptime</code>	✓	✓ <code>--min-uptime</code>
<code>--restart-delay</code>	✓	✓ <code>--restart-delay</code>
<code>--no-autorestart</code>	✓	✓ <code>--no-auto-restart</code>
<code>--merge-logs</code>	✓	✓ <code>--merge-logs</code>
<code>--kill-timeout</code>	✓	✓ <code>--kill-timeout</code>
<code>--log-date-format</code>	✓	⚠ Config only
<code>--out / --error</code>	✓	⚠ Config only (<code>out_file</code> , <code>err_file</code>)
<code>--ignore-watch</code>	✓	⚠ Config only (<code>ignore_watch</code>)
<code>--wait-ready</code>	✓	✗
<code>--listen-timeout</code>	✓	✗
<code>--shutdown-with-message</code>	✓	✗

Ecosystem config compatibility

Warden reads the same ecosystem file formats PM2 uses (`ecosystem.config.js` , `ecosystem.config.json` , and other `*.config.js` files). Field names follow PM2 conventions:

```
// Works with both PM2 and Warden
module.exports = {
  apps: [
    {
      name: 'api',
      script: './src/api.js',
      instances: 2,
      exec_mode: 'cluster',
      env: { NODE_ENV: 'production', PORT: '3000' },
      max_memory_restart: '500M',
      watch: ['./src'],
      ignore_watch: ['node_modules'],
      cron_restart: '0 3 * * *',
      interpreter: 'node',
      max_restarts: 16,
      min_uptime: '1s',
      autorestart: true,
      merge_logs: true,
      kill_timeout: 5000,
    },
  ],
};
```

Migration tip: If you already have a PM2 ecosystem file, you can usually point Warden at it unchanged:

```
# Before
pm2 start ecosystem.config.js

# After
warden start ecosystem.config.js
```

File locations

Purpose	PM2	Warden
Home directory	~/.pm2/	~/.warden/
Saved process list	~/.pm2/dump.pm2	~/.warden/dump.json
Daemon PID	~/.pm2/pm2.pid	~/.warden/daemon.pid
IPC socket	~/.pm2/rpc.sock / pub.sock	~/.warden/daemon.sock
Logs	~/.pm2/logs/	~/.warden/logs/<name>-<id>/
Metrics	External (PM2 Plus)	~/.warden/metrics.db (SQLite)

Observability

PM2

- **Terminal:** `pm2 monit`, `pm2 list` with inline CPU/memory
- **Cloud:** PM2 Plus — web dashboard, alerts, multi-server (paid subscription)
- **Prometheus:** Requires community modules or custom integration
- **REST API:** Limited; full API tied to PM2 Plus

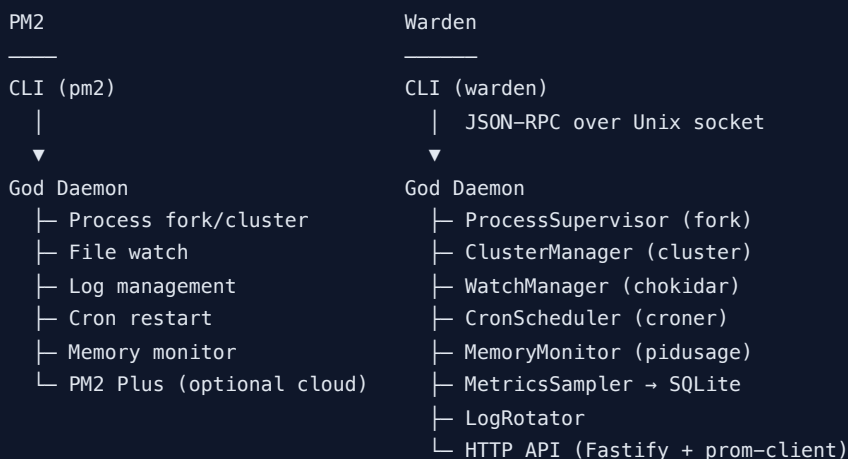
Warden

- **Terminal:** `warden monit`, `warden list`, `warden describe`
- **HTTP API:** Built-in on port 9615 — `/processes`, `/processes/:id/metrics?range=1h`, `/summary`, `/health`
- **Prometheus:** Native `/metrics` endpoint with `warden_process_*` gauges
- **Time-series:** SQLite database with 5-second sampling and 24-hour retention

Example Prometheus scrape config for Warden:

```
scrape_configs:
  - job_name: warden
    static_configs:
      - targets: ['127.0.0.1:9615']
```

Architecture



Both use a long-running **god daemon** that outlives the CLI. The CLI connects, sends commands, and exits — managed processes keep running.

When to choose PM2

- You need **PM2 Plus** cloud monitoring, alerts, or multi-server fleet management
- You rely on `pm2 deploy` for Git-based deployment workflows
- You use the **PM2 module ecosystem** (`pm2-logrotate` , custom modules)
- You need `pm2 attach` , `pm2 trigger` , or **JSON output** (`pm2 jlist`) for automation
- You want an **official Docker image** with PM2 pre-installed
- You need **Windows service** integration out of the box

When to choose Warden

- You want a **fully self-hosted** solution with no cloud dependency or subscription
- You need **native Prometheus** metrics and a **built-in HTTP API** without extra modules
- You want **local time-series metrics** (SQLite) for debugging and short-term analysis
- You prefer an **MIT-licensed**, TypeScript codebase you can fork and extend
- You're running a **single server or small fleet** and terminal + Prometheus/Grafana is enough
- You're migrating from PM2 and your workflow is: start → logs → monit → save → startup

Quick migration guide (PM2 → Warden)

```
# 1. Save your current PM2 process list (for reference)
pm2 save
pm2 jlist > pm2-backup.json

# 2. Stop PM2 (optional – they can coexist on different sockets)
pm2 kill

# 3. Install Warden
npm install -g @roshan-gamage/warden

# 4. Start your apps with the same ecosystem file
warden start ecosystem.config.js

# 5. Persist and enable boot startup
warden save
warden startup    # follow printed instructions for systemd / launchd

# 6. Verify
warden list
warden ping
curl http://127.0.0.1:9615/health
```

If you start processes individually instead of via an ecosystem file, recreate them with equivalent `warden start` flags (see CLI mapping above), then `warden save` .

Summary

Dimension	Winner
Core process management (fork, cluster, restart, logs)	Tie — feature parity for everyday use
Ecosystem config compatibility	Tie — same file format
Built-in observability (API, Prometheus, metrics DB)	Warden
Cloud monitoring & fleet management	PM2 (PM2 Plus)
Module / plugin ecosystem	PM2
Deployment automation	PM2 (<code>pm2 deploy</code>)
Licensing & self-hosting	Warden (MIT, no cloud lock-in)
Maturity & community	PM2 — larger user base, longer track record

Warden covers the PM2 workflow most teams actually use — start, supervise, log, monitor, persist, and boot — while adding first-class telemetry. PM2 remains the better choice when you need cloud dashboards, deployment tooling, or the module ecosystem.